

EX 2 MULTI-LAYER PERCEPTRON (MLP) FOR CLASSIFICATION

DATE:

Problem Statement:

Develop a Multi-Layer Perceptron (MLP) for a simple classification task. Experiment with different numbers of hidden layers and activation functions, and evaluate the model's performance using accuracy and loss.

Suggested Dataset: Iris Dataset

```
import numpy as np

import matplotlib.pyplot as plt


from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler


from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense


# Load Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

classes = iris.target_names


# Scale data
scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)
```

```
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
# Create MLP
```

```
def create_mlp(hidden_layers, activation):
```

```
    model = Sequential()
```

```
    model.add(
```

```
        Dense(
```

```
            hidden_layers[0],
```

```
            input_shape=(4,),
```

```
            activation=activation
```

```
        )
```

```
    )
```

```
    for units in hidden_layers[1:]:
```

```
        model.add(
```

```
            Dense(units, activation=activation)
```

```
        )
```

```
model.add(
    Dense(3, activation="softmax")
)
```

```
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

```
return model
```

```
# Configurations
```

```
hidden_layer_configs = [
    [8],
    [16, 8],
    [32, 16, 8]
]
```

```
activations = [
    "relu",
    "tanh",
    "sigmoid"
]
```

```
results = []
```

```
# Train models
```

```
for hidden_layers in hidden_layer_configs:
```

```
    for activation in activations:
```

```
        print("\n-----")
```

```
        print("Hidden Layers:", hidden_layers)
```

```
        print("Activation:", activation)
```

```
        print("-----")
```

```
        model = create_mlp(
```

```
            hidden_layers,
```

```
            activation
```

```
        )
```

```
        history = model.fit(
```

```
            X_train,
```

```
            y_train,
```

```
            epochs=30,
```

```
            batch_size=8,
```

```
            validation_split=0.1,
```

```
            verbose=0
```

```
        )
```

```
        loss, accuracy = model.evaluate(
```

```
            X_test,
```

```
            y_test,
```

```
            verbose=0
```

```
)
```

```
print("Test Accuracy:", round(accuracy * 100, 2), "%")
```

```
print("Test Loss:", round(loss, 4))
```

```
results.append(
```

```
(
```

```
    hidden_layers,
```

```
    activation,
```

```
    accuracy,
```

```
    loss
```

```
)
```

```
)
```

```
# Display results
```

```
print("\n\nFINAL RESULTS")
```

```
print("-----")
```

```
for result in results:
```

```
    hidden_layers, activation, accuracy, loss = result
```

```
print(
```

```
    "Layers:",
```

```
    hidden_layers,
```

```
    "| Activation:",
```

```
    activation,
```

```
"| Accuracy:",  
round(accuracy * 100, 2),  
"%",  
"| Loss:",  
round(loss, 4)  
)
```

```
# Train final model for prediction
```

```
model = create_mlp(  
    [32, 16, 8],  
    "relu"  
)
```

```
history = model.fit(  
    X_train,  
    y_train,  
    epochs=30,  
    batch_size=8,  
    validation_split=0.1,  
    verbose=0  
)
```

```
# Plot accuracy
```

```
plt.figure(figsize=(7, 5))
```

```
plt.plot(  
    history.history["accuracy"],
```

```
        label="Training Accuracy"
    )

plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy"
)
```

```
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("MLP Accuracy")
plt.legend()
plt.grid()
plt.show()
```

Plot loss

```
plt.figure(figsize=(7, 5))

plt.plot(
    history.history["loss"],
    label="Training Loss"
)
```

```
plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)
```

```
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("MLP Loss")
plt.legend()
plt.grid()
plt.show()
```

```
# User input prediction
print("\nEnter flower measurements")
```

```
sepal_length = float(input("Sepal Length (cm): "))
sepal_width = float(input("Sepal Width (cm): "))
petal_length = float(input("Petal Length (cm): "))
petal_width = float(input("Petal Width (cm): "))
```

```
user_input = np.array([
    [
        sepal_length,
        sepal_width,
        petal_length,
        petal_width
    ]
])
```

```
user_input_scaled = scaler.transform(user_input)
```

```
prediction = model.predict(
    user_input_scaled,
    verbose=0
)
```

```
predicted_index = np.argmax(prediction)
```

```
print("\nPredicted Iris Species:")
print(classes[predicted_index])
```

Hidden Layers: [8]

Activation: relu

C:\Users\Balag\anaconda3\Lib\site-packages\keras\src\layers\core\dense.py:107:
UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using
Sequential models, prefer using an `Input(shape)` object as the first layer in the model
instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for
TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use
WSL2 or the TensorFlow-DirectML plugin.

Test Accuracy: 76.67 %

Test Loss: 0.4694

Hidden Layers: [8]

Activation: tanh

Test Accuracy: 80.0 %

Test Loss: 0.4102

Hidden Layers: [8]

Activation: sigmoid

Test Accuracy: 80.0 %

Test Loss: 0.6485

Hidden Layers: [16, 8]

Activation: relu

Test Accuracy: 90.0 %

Test Loss: 0.2655

Hidden Layers: [16, 8]

Activation: tanh

Test Accuracy: 93.33 %

Test Loss: 0.2867

Hidden Layers: [16, 8]

Activation: sigmoid

Test Accuracy: 76.67 %

Test Loss: 0.7091

Hidden Layers: [32, 16, 8]

Activation: relu

Test Accuracy: 93.33 %

Test Loss: 0.1388

Hidden Layers: [32, 16, 8]

Activation: tanh

Test Accuracy: 96.67 %

Test Loss: 0.1555

Hidden Layers: [32, 16, 8]

Activation: sigmoid

Test Accuracy: 73.33 %

Test Loss: 0.7092

FINAL RESULTS

Layers: [8] | Activation: relu | Accuracy: 76.67 % | Loss: 0.4694

Layers: [8] | Activation: tanh | Accuracy: 80.0 % | Loss: 0.4102

Layers: [8] | Activation: sigmoid | Accuracy: 80.0 % | Loss: 0.6485

Layers: [16, 8] | Activation: relu | Accuracy: 90.0 % | Loss: 0.2655

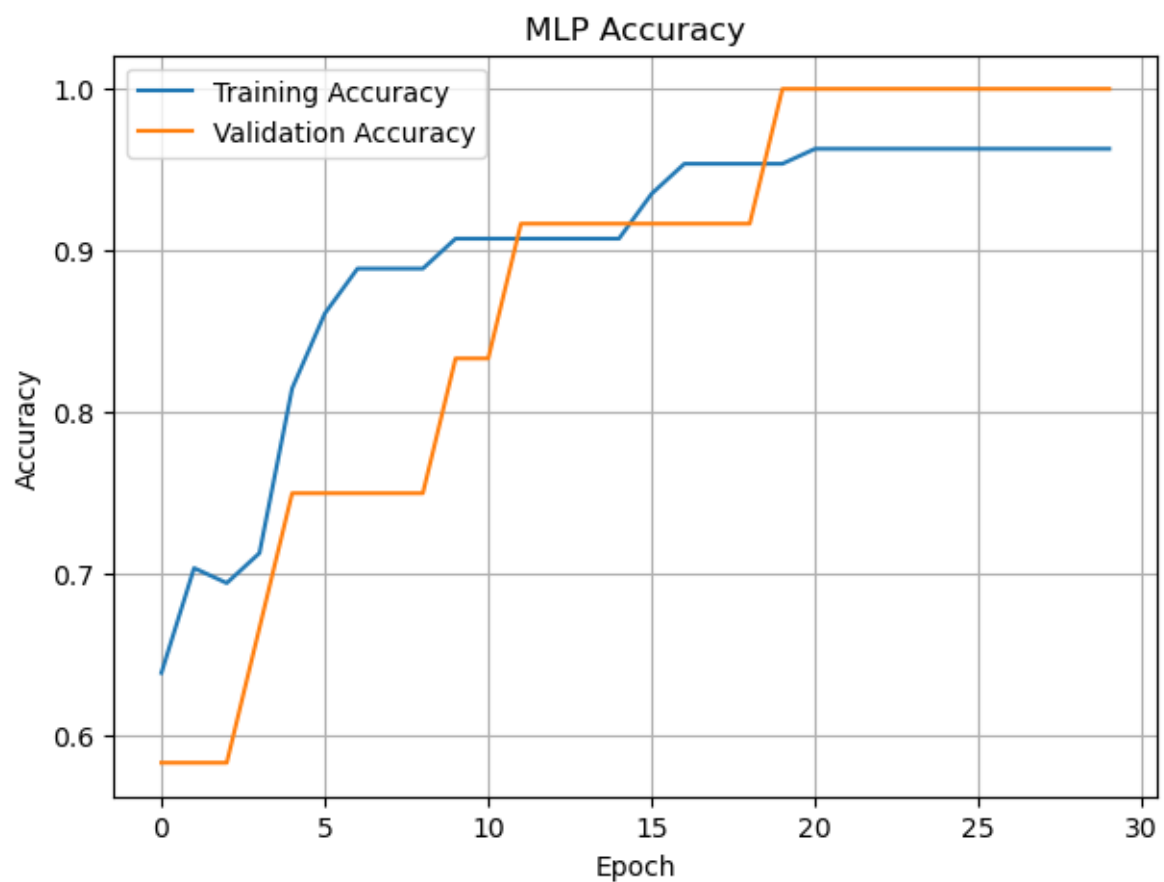
Layers: [16, 8] | Activation: tanh | Accuracy: 93.33 % | Loss: 0.2867

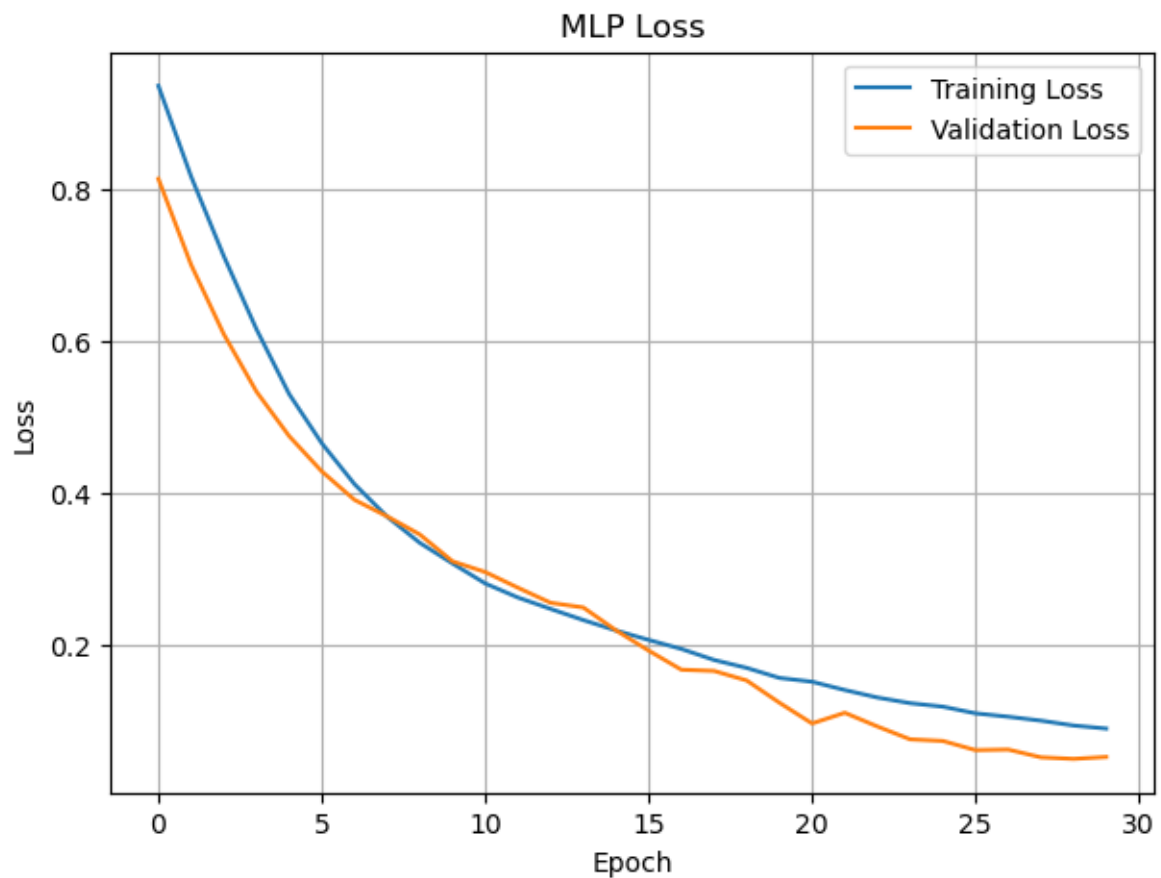
Layers: [16, 8] | Activation: sigmoid | Accuracy: 76.67 % | Loss: 0.7091

Layers: [32, 16, 8] | Activation: relu | Accuracy: 93.33 % | Loss: 0.1388

Layers: [32, 16, 8] | Activation: tanh | Accuracy: 96.67 % | Loss: 0.1555

Layers: [32, 16, 8] | Activation: sigmoid | Accuracy: 73.33 % | Loss: 0.7092





Enter flower measurements

Sepal Length (cm): 5

Sepal Width (cm): 2

Petal Length (cm): 8

Petal Width (cm): 3

Predicted Iris Species:

virginica